

Dobot Magician Calibration and Setup Guide

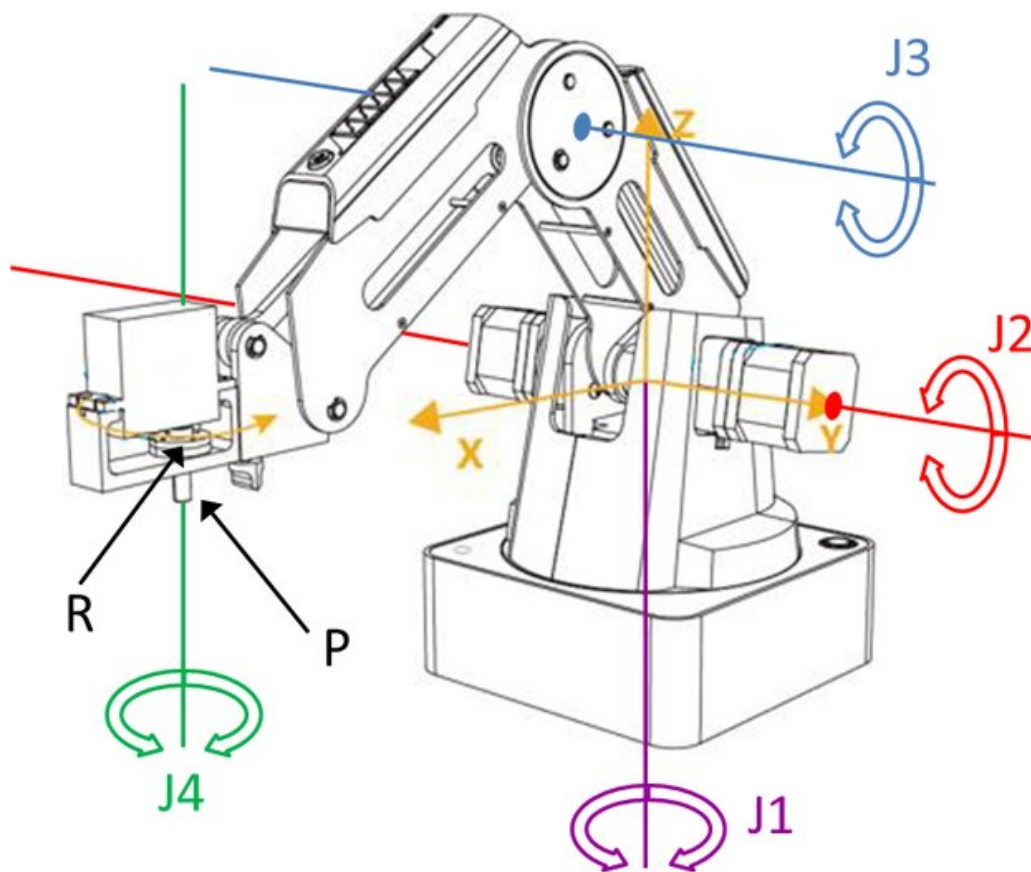
ME403 Introduction to Robotics

1 Purpose

This guide explains the minimum setup required before running Python scripts on a real Dobot Magician in ME403 labs. The most important prerequisite is **calibrating (homing)** with original Dobot software before Python control.

2 Robot Joint Reference

The Magician is a 4-axis robot with revolute joints $J1$ through $J4$. The diagram below is useful when interpreting joint commands and measured end-effector pose in labs.



Joint reminders

- J1: base rotation
- J2, J3: arm elevation links (parallelogram structure keeps tool orientation behavior consistent)
- J4: end rotation axis

3 Critical Calibration Rule

Always calibrate/home the Magician in original Dobot software before Python runs.

- After power-on, the robot does not reliably know its working reference frame.
- Homing re-establishes reference and significantly improves repeatability and parity with simulation.
- Dobot software and Python scripts cannot control the same robot at the same time.

4 Download the Original Software

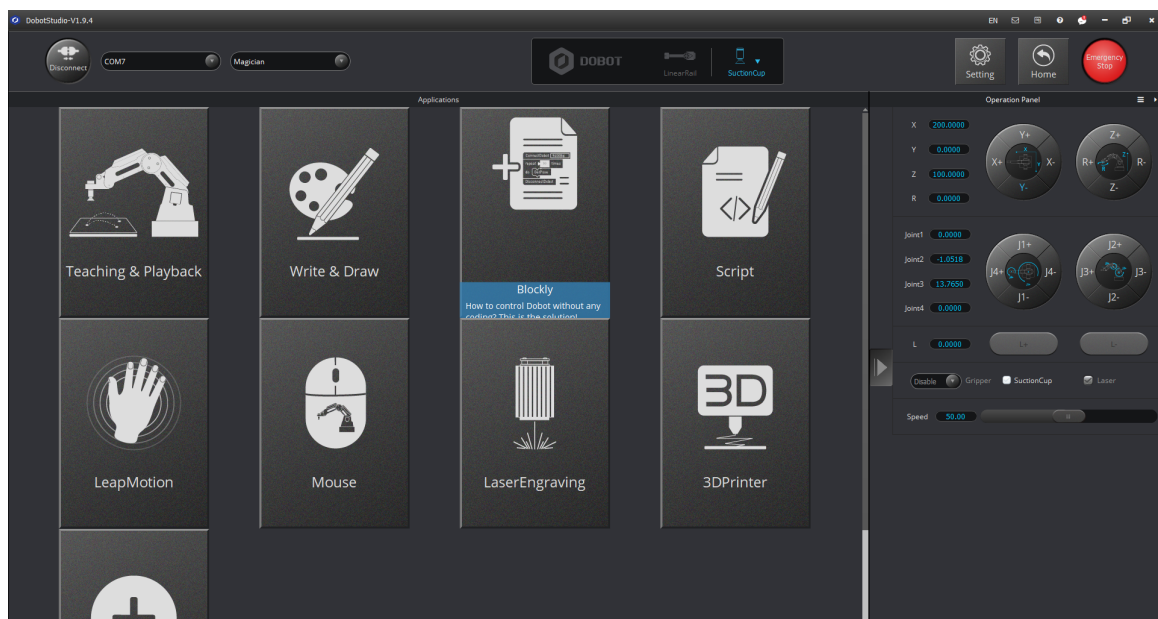
Use the official Dobot Download Center:

<https://www.dobot-robots.com/service/download-center>

In the **Control Software** category, download and install **DobotStudio (DOBOT Magician) v1.9.4** (released Jan 10, 2022).

5 Homing Workflow (Recommended)

1. Connect Magician power and USB to your laptop.
2. Launch DobotStudio.
3. Click **Connect** and confirm communication is healthy.
4. Run **Home/Calibration** and wait for completion.
5. In end-effector configuration, select **None**.
6. In advanced end-effector options, set XYZ bias to 0, 0, 0.
7. Click **Disconnect** (or close the application).
8. Start your Python lab scripts.



6 Pre-run Checklist for Python

Check	Expected state before running Python
USB + Power	Robot powers on and USB serial device appears
Calibration	Home completed in DobotStudio
Controller ownership	Dobot software disconnected/closed before Python
End-effector config	Type <code>None</code> , advanced XYZ bias 0,0,0
Linux permissions	User in <code>dialout</code> group (Linux only)
Sanity probe	<code>python3 scripts/check_magician.py -connect</code> succeeds

7 Common Failure Modes

Large real-vs-sim mismatch

Most commonly caused by skipping calibration/homing before parity or lab runs. Re-home in Dobot software, disconnect it, then rerun.

Serial device not found

Check USB/power, close Dobot software, and verify Linux serial permissions.

Robot does not move as expected after long session

Re-run software homing and restart the Python script to re-initialize state.

8 Setting End-Effector Type from Python

Once the robot has been homed in DobotStudio, the TCP offset can be changed from Python for all subsequent sessions.

Why this matters

The Magician firmware stores an (o_x, o_y, o_z) bias that shifts every `GetPose` result. DobotStudio’s “end-effector type” selector writes this bias to the robot’s EEPROM via Protocol ID 60. The same command is available from Python through `set_tool_offset()` in `utils.py`.

End-effector preset table

Preset	TCP offset (mm)	Home position at (0,0,0,0)
<code>none</code> (bare flange)	(0, 0, 0)	(147, 0, 135) mm
<code>motor</code> (motor flange)	(60, 0, 0)	(207, 0, 135) mm
<code>suction</code> (physical cup tip)	(60, 0, -70)	(207, 0, 65) mm

Note on DobotStudio: The factory “suction cup” preset in DobotStudio uses TCP offset (+60,0,0) mm — same as the motor flange — because it reports the motor-shaft TCP, not the physical cup tip. Use the `motor` preset (or `set_tool_offset(bot, 60, 0, 0)`) when you want `GetPose` to match DobotStudio readings with the suction cup attached. Use the `suction` preset to model the physical cup tip for accurate FK/IK.

Python API

```
from utils import MAGICIAN_TOOL_OFFSETS, set_tool_offset,
    get_tool_offset

bot = setup()    # connect to robot

# Read what the firmware currently has
ox, oy, oz = get_tool_offset(bot)
print(f"Current TCP offset: ({ox:.1f}, {oy:.1f}, {oz:.1f}) mm")

# Set to bare flange (required for FK labs)
set_tool_offset(bot, *MAGICIAN_TOOL_OFFSETS["none"])    # writes (0, 0,
    0)

# Match DobotStudio "suction cup" factory preset (motor-shaft TCP)
set_tool_offset(bot, *MAGICIAN_TOOL_OFFSETS["motor"])    # writes (60,
    0, 0)
```

The setting is stored in EEPROM and persists after power-off.

Simulation alignment

Keep the simulation's DOBOT_EE environment variable in sync with the real robot's TCP offset so that FK/IK measurement errors reflect kinematics, not a mode mismatch:

```
# Simulation in bare-flange mode (FK labs default)
DOBOT_EE=None DOBOT_SIMULATION=1 python3 interface.py
```

9 ME403 Command Examples

From ME403_LabFiles/ root:

```
python3 scripts/check_magician.py
python3 scripts/check_magician.py --connect
```

Run Lab 1 interface:

```
cd labs/lab01_fk
python3 interface.py
```

10 Summary

1. **Home once** in DobotStudio after power-on.
2. **Disconnect** DobotStudio before running Python.
3. **Set TCP offset** — either in DobotStudio (type = None) or from Python: `set_tool_offset(bot, *MAGICIAN_TOOL_OFFSETS["none"])`.
4. For FK labs, use **bare-flange mode (none)** on both real robot and simulation (`DOBOT_EE=None`).

This workflow eliminates the most common setup and parity issues in Magician labs.